

brown open silicon

designing open chips. together.

agenda



icebreaker

digital design
process
(xinting)

tools introduction & reverse
engineering
(anhang)



“i like doing layout manually”



**Imagine you were a
professor at Brown. What
kind of course would you
teach?**

digital **design** tutorial

from logic gates to silicon. **together.**

agenda



what we'll cover today

01 intro to digital design

what is HDL? how does logic become silicon?

02 verilog fundamentals

modules, ports, assign, always blocks

03 HDLBits exercises

hands-on: gates, mux, flip-flops at hdlbits.01xz.net

04 Exercise

Round Robin arbiter

what is digital design?



digital design is the process of creating circuits that process information using **binary logic (0s and 1s)**

everything starts with

logic gates → **circuits** → **processors**

describe hardware using

HDL (Hardware Description Language)

then synthesize it into real gates that go on a chip!



the design flow



how does Verilog code become a real chip?

1. **RTL Design** — write your hardware in Verilog/VHDL
2. **Simulation** — verify behavior with testbenches
3. **Synthesis** — RTL → gate-level netlist (Yosys / DC)
4. **Place & Route** — map gates onto real physical locations
5. **Tapeout / Bitstream** — send to fab or program an FPGA

what is verilog?



Verilog is a Hardware Description Language (HDL)

unlike regular code, Verilog **describes hardware**
not instructions for a CPU to execute sequentially

key insight: everything runs **concurrently**
multiple blocks of logic operate at the same time

verilog: module structure



every design is a **module** with inputs and outputs

```
module my_and(  
    input  wire a, b,  
    output wire out  
);
```

```
    assign out = a & b;    // continuous assign
```

```
endmodule
```

module / **endmodule** define the boundary of your hardware block



assign & always blocks

assign — combinational logic (no clock)

```
assign sum = a ^ b;           // XOR
assign carry = a & b;         // AND
```

always @(posedge clk) — sequential (clocked)

```
always @(posedge clk) begin
    if (reset) q <= 0;
    else      q <= d;
end
```

use **=** inside combinational always, **<=** inside sequential always

hdlbits: basic gates



head to **hdlbits.01xz.net** → Verilog Language → Basics

try these problems:

Wire — assign out = in;

AND gate — assign out = a & b;

OR gate — assign out = a | b;

XOR gate — assign out = a ^ b;

these are all **combinational** — output depends only on current inputs



hdlbits: multiplexers



a **2-to-1 mux** selects between two inputs:

```
module mux2to1(  
    input  wire a, b, sel,  
    output wire out  
);  
    assign out = sel ? b : a; // if sel=1 pick b, else a  
endmodule
```

challenge: try **Mux2to1v** (4-bit wide) on HDLBits!
Verilog Language → Mux and DeMux

hdlbits: flip-flop



a **D flip-flop** stores one bit of state:

```
module dff(  
    input  wire clk, d,  
    output reg  q  
);  
    always @(posedge clk)  
        q <= d; // capture d on rising clock edge  
endmodule
```

find it at: Verilog Language → Procedures → **Always FF**

synthesis: concept



synthesis translates your Verilog into a **gate-level netlist**

your RTL (behavioral description)



AND gates, OR gates, flip-flops...



physical transistors on silicon

the synthesis tool maps your logic to cells in a **standard cell library**
each cell is a pre-characterized logic gate with known area, power,
timing



synthesis: tools & flow



open-source stack we use at Brown Silicon:

Yosys — open-source synthesis (RTL → netlist)

OpenROAD — place & route after synthesis

Librelane — full RTL-to-GDSII flow (uses both)

```
# synthesize a Verilog file with Yosys
```

```
yosys -p "synth -top top; write_verilog out.v" design.v
```

industry tools: Synopsys Design Compiler, Cadence Genus

what is an fpga?



FPGA = Field-Programmable Gate Array

a chip full of **reconfigurable logic blocks** connected by
programmable wiring

you write Verilog → synthesize → generate a **bitstream**
the bitstream programs the FPGA to behave like your circuit

asic vs fpga



ASIC (Application-Specific IC)

custom chip manufactured for one purpose

fast + power-efficient + expensive to fabricate

examples: Apple M-series, Nvidia GPUs, SkyWater shuttle chips

FPGA (Field-Programmable Gate Array)

reprogrammable chip for rapid prototyping

slower + more power, but no fab cost or wait

great for verifying RTL before committing to tapeout



try it yourself!



practice Verilog right now in your browser:

hdlbits.01xz.net

recommended starting path:

Getting Started → Verilog Language → Basics → Mux → Procedures

next session: **OpenLane walkthrough + your first synthesized design**
questions?

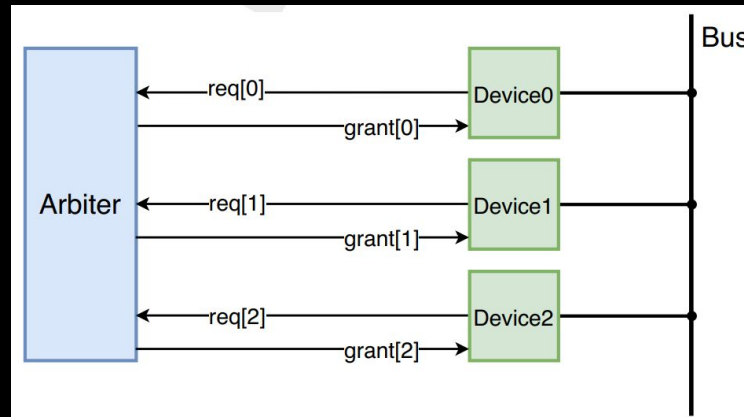
Exercise: round robin arbiter



exercise

design a simplified FSM-based bus arbiter
using SystemVerilog

topics: FSMs | request/grant protocol | testbenches



what is a bus arbiter?



multiple devices share a single **bus** — but only one can use it at a time

the arbiter manages access via **request** and **grant** signals

in this lab: $n = 3$ **devices** (Device0, Device1, Device2)

protocol:

1. device asserts **req[i]** to request bus access
2. arbiter asserts **grant[i]** on the next posedge clk
3. device holds req until grant received; arbiter holds grant until req drops





round robin strategy

round robin = each device takes turns in a fixed cyclic order

the pattern cycles:

Device2 → **Device1** → **Device0** → Device2 → ...

key rules:

1. if DeviceX's turn but $\text{req}[X] = 0$, **skip to Device(X+1)**
2. grant is sent on posedge clk, holds until req drops or next cycle
3. reset is **asynchronous** — goes to IDLE and clears all grants immediately

arbiter fsm: states



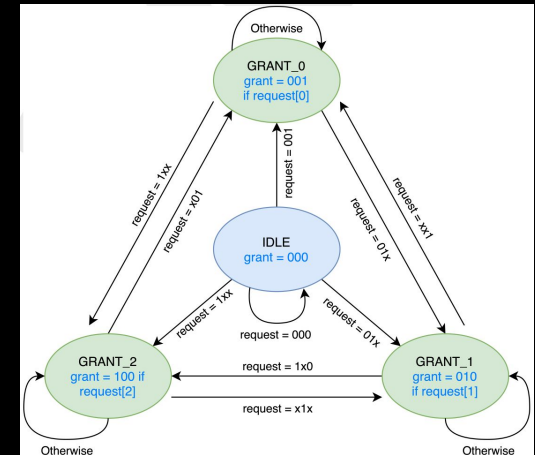
the arbiter has **4 states** in its FSM:

IDLE — grant = 000, no device is granted
waits for any request; priority: Device2 > Device1 > Device0

GRANT_0 — grant = 001, Device0 has the bus
stays if req[0] still high (“otherwise”), else back to IDLE

GRANT_1 — grant = 010, Device1 has the bus
stays if req[1] still high (“otherwise”), else back to IDLE

GRANT_2 — grant = 100, Device2 has the bus
stays if req[2] still high (“otherwise”), else back to IDLE



arbiter fsm: transitions



from **IDLE** (grant = 000):

```
req = 1xx -> GRANT_2    (Device2 has highest priority)
req = 01x -> GRANT_1
req = 001 -> GRANT_0
req = 000 -> IDLE      (stay idle)
```

from **GRANT_2** (grant = 100):

```
req = x1x -> GRANT_1    (next in rotation)
req = 1x0 -> stay       (still Device2's turn: "otherwise")
req = 001 -> GRANT_0
```

transitions from GRANT_1 and GRANT_0 follow the same round-robin
pattern

see the full FSM diagram in the lab handout (Figure 2)

starter code: arbiter.sv



fill in the **TODO** sections in **arbiter.sv**

```
module arbiter(  
    input  logic      clk, reset,  
    input  logic [2:0] req,  
    output logic [2:0] grant  
);  
    typedef enum logic [1:0] {  
        IDLE, GRANT_0, GRANT_1, GRANT_2  
    } state_t;  
    state_t state, next_state;  
    // TODO: implement next_state logic + output logic + reset  
endmodule
```

tip: use **always_ff** for state register and **always_comb** for
next-state/output logic

testbench: arbiter_test.sv



a good testbench must have **coverage** AND be **fast**

5 provided test cases:

reset behavior, single requests, simultaneous requests,
priority ordering, grant hold/release

your random test cases (TODO):

```
// randomize req and check grant follows FSM transitions
repeat(N) begin
    req = $urandom_range(0, 7); // 3-bit random
    @(posedge clk);
    assert(grant == expected) else $error(...);
end
```

extend simulation time in Settings if your testbench needs more than
1000ns



common pitfalls to avoid

1. blocking vs non-blocking assignments

always_ff → use `<=` | always_comb → use `=`

2. incomplete case statements

always use a **default** branch to avoid inferred latches

3. forgetting asynchronous reset

```
always_ff @(posedge clk or posedge reset)
    if (reset) state <= IDLE;
```

4. forgetting “otherwise” transitions

in GRANT_X: if req[X] is still high, **stay in current state**

5. simulation time too short

Tools → Settings → Simulation → increase **xsim.simulate.runtime**



tools & reverse engineering

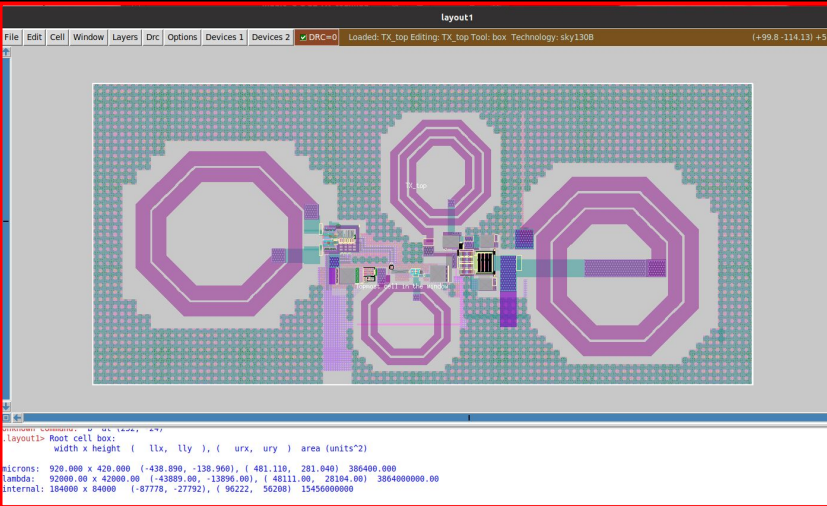
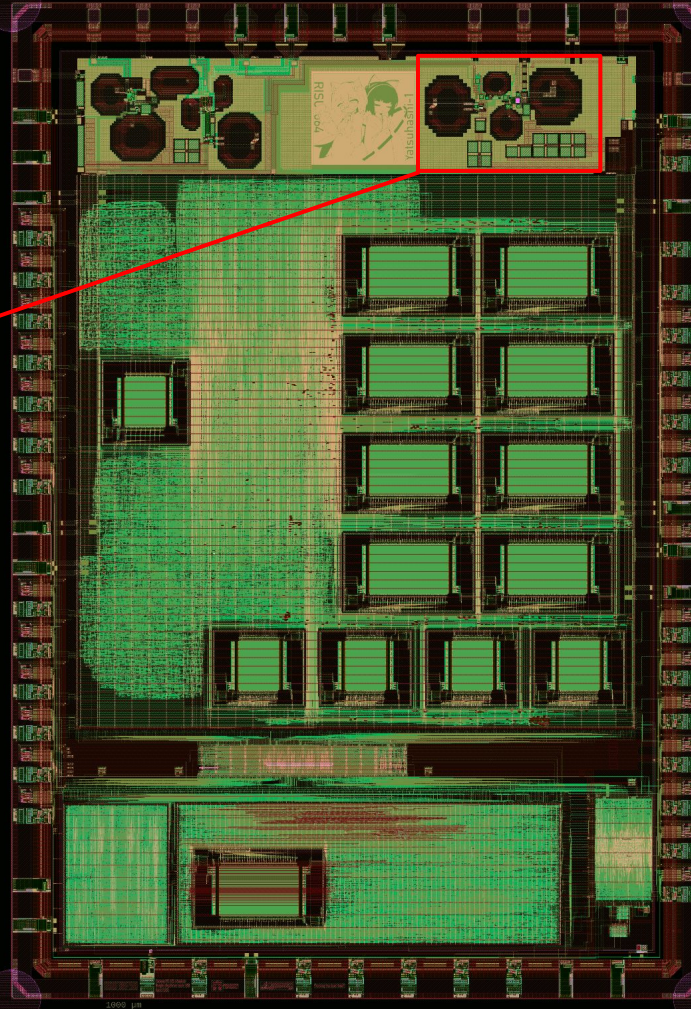
Advanced projects

RF Design: 5GHz TRX



5GHz
TRX

Dual-Issue
Risc-V



Stacked Class-E Power Amplifier

Advanced projects

RF Design: 5GHz TRX



5GHz
TRX

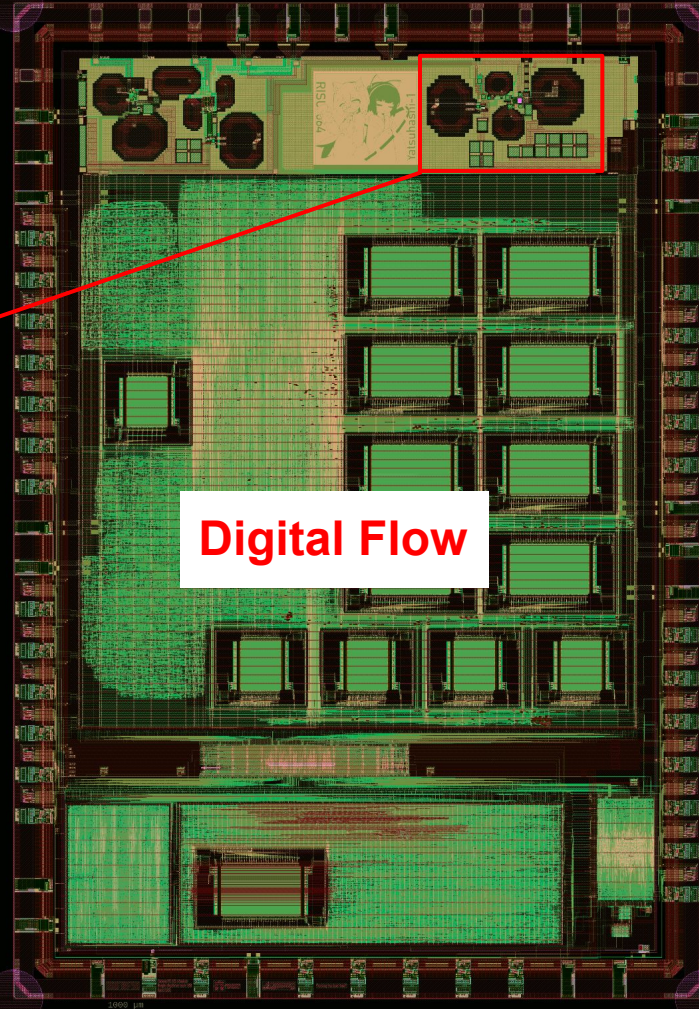
Dual-Issue
Risc-V

Digital Flow

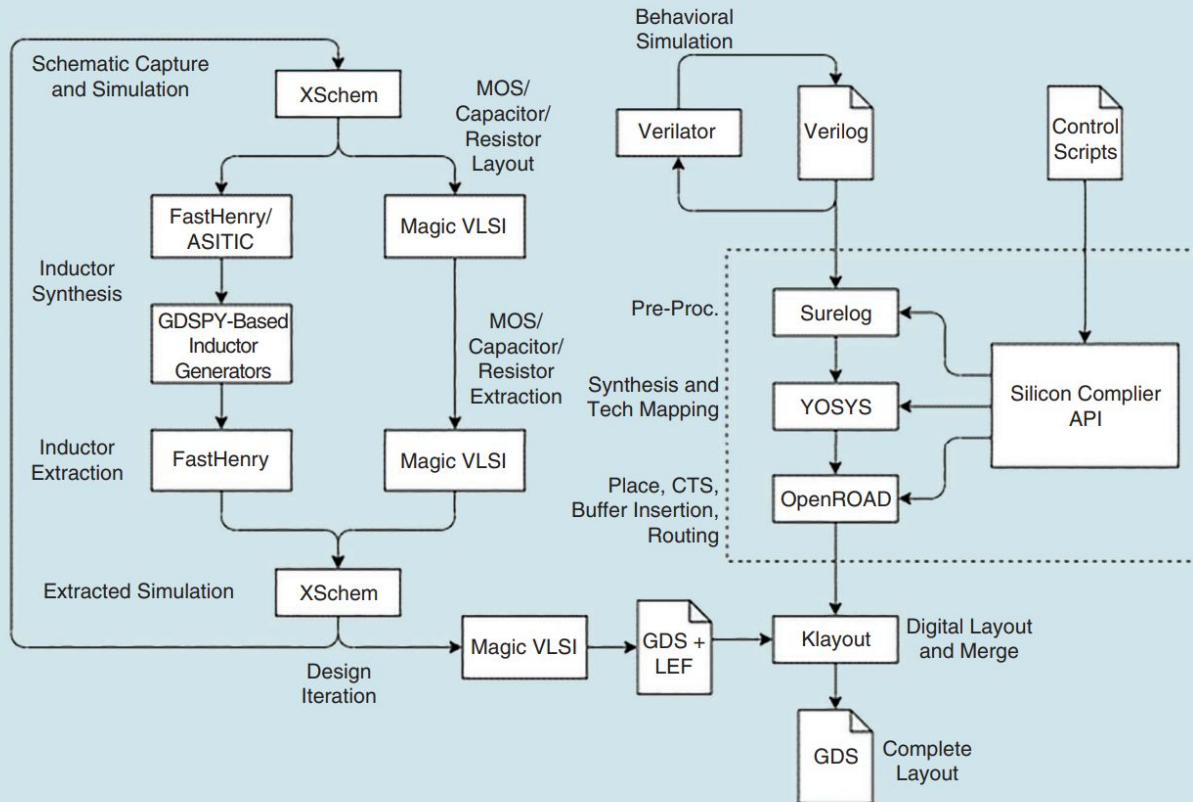
Custom
Analog

Magnetics

Stacked Class-E Power Amplifier



Design Flow Overview



General Digital/Analog Design Tools



iic-jku/IIC-OSIC-TOOLS



IIC-OSIC-TOOLS is an all-in-one Docker image for SKY130/GF180/IHP130-based analog and digital chip design. AMD64 and ARM64 are natively supported.



0 Contributors



21 Issues



2 Discussions



526 Stars



89 Forks



Schematic Capture:	XSchem
Simulation:	ngSpice, Xyce
Layout Design:	kLayout, MAGIC VLSI
Digital Flow:	OpenROAD

Mixed Signal Simulation



Minimum ngSpice Mixed-Signal Example

Author: Anhang Li (anhangli@umich.edu)

Overview

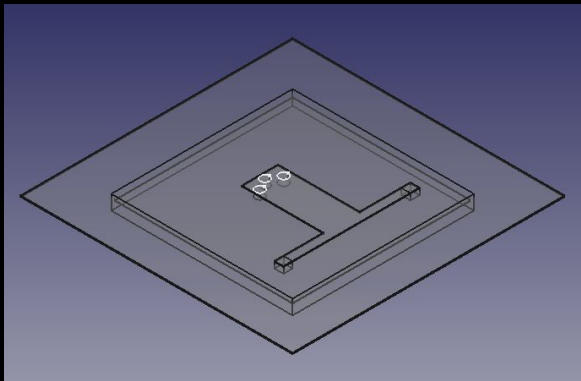
This is a minimum example of using ngSpice to simulate a mixed-signal circuit. Based on [Carsten Wulff's tutorial](#) but further simplified for lecture purposes.

The verilog module in this example is a simple 5-bit counter.

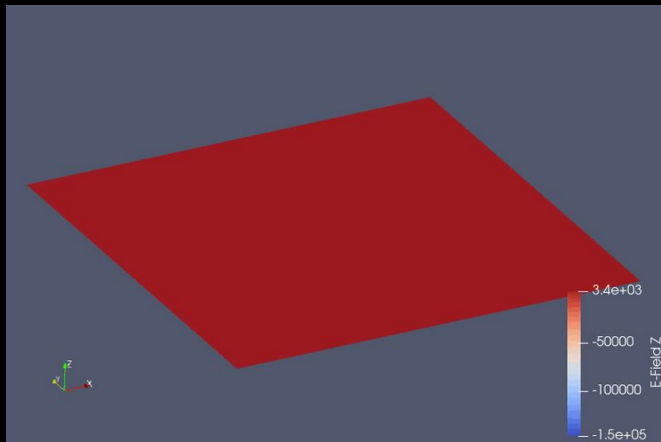


ngSpice natively supports simulating analog circuits with digital behavioural modules.

EM Simulation



**EM Structure
Modeling in FreeCAD**



**Simulation in OpenFEM
Visualization in Paraview**



FEM Simulator:
FDTD Simulator:
Inductance solver:

OpenFEM, Elmer FEM
OpenEMS, MEEP
FastHenry

Other Resources



**eFabless Github Archive
Complete SoC Design Files**



**Prof. Peter Kinget's
VLSI Design Lab**



**SSCS Code-a-Chip
Archive**



thank you for coming!



you (an expert in verilog)